

1 Introduction

1.1 Motivation

Dependently typed languages such as Lean, Agda, and Rocq serve as both programming languages and proof assistants. Unlike ordinary compilers, their type checkers must execute arbitrary programs at compile time — normalising terms, checking definitional equality, and resolving implicit arguments. This process, called *elaboration*, is the most expensive phase of compilation.

Elaboration is typically performed in batch mode. When a user edits a single definition in a file, the elaborator re-checks every subsequent definition from that point onwards. In practice, I found that even a small edit to one line of a proof could trigger seconds of re-elaboration across hundreds of unrelated definitions. In an interactive setting — where a proof assistant is expected to provide rapid feedback on each keystroke — this is disruptive.

The central observation is that most of this re-checking is redundant. If one definition changes but its type is unaffected, then definitions that depend only on its type need not be re-checked at all. What is needed is a way to track which results depend on which, and recompute only what has actually been invalidated.

This is a well-studied problem in a different guise: it is exactly what build systems do. *GNU Make* tracks file modification times to avoid rebuilding targets whose dependencies have not changed. The Haskell library *Shake* generalises this with verifying traces and dynamic dependencies. The framework of Mokhov et al. [1] shows that Make, Shake, and many other build systems are instances of a single polymorphic type, parameterised by the class of effects a task is allowed to perform.

This project applies that framework to elaboration. The elaborator decomposes its work into fine-grained queries — “what is the type of this constant?”, “what are the errors in this definition?” — each of which is memoised by a Shake-based build system. When a definition changes, only the queries that depend on it are reconsidered; and if the result of a recomputed query is unchanged, its dependents are not recomputed at all. The implementation is written in Lean 4, and built on a formalisation of the build system framework in the same language.

1.2 Contributions

- An incremental, query-based elaborator for a dependently typed language, supporting dependent function types, universe polymorphism, and inductive types with recursors.
- A formalisation of the “Build Systems à la Carte” framework in Lean 4, with intrinsic correctness guarantees.
- An evaluation comparing incremental re-elaboration against batch re-elaboration on a standard library.

1.3 Overview

Section 2 introduces the background material: the build system framework of Mokhov et al. and our refinements to it, the dependent type theory we implement, bidirectional type checking, normalisation by evaluation, and related work. Section 3 describes the elaborator’s architecture and each component of the pipeline: parsing, type checking, evaluation, conversion checking, inductive type elaboration, and the incremental build and language server infrastructure. Section 4 presents measurements of correctness, batch elaboration performance, conversion checking speed, and the benefit of incrementality. Section 5 summarises the results and discusses future work.

2 Preparation

2.1 Build systems à la carte

Mokhov et al. [1] observe that build systems and spreadsheet engines solve the same problem: given a set of tasks with dependencies, bring outputs up to date with respect to inputs. They introduce a common framework that captures *Make*, *Shake*, *Bazel*, *Excel*, and others as instances of a single polymorphic type.

The central abstraction is the *task*, which describes how to compute a value from its dependencies. A task is polymorphic in an effect f :

$$\text{Task } c \, k \, v = \forall f. [c \, f] \Rightarrow (k \rightarrow f \, v) \rightarrow f \, v$$

A *task* receives a *fetch* callback that retrieves the value of any key, and produces its own value in the effect f , chosen by the build system. For example, the build system may choose f to log all the queries that were fetched by this task in some mutable state, in order to do some memoisation later. The constraint c on f determines what the task can do with the results of its fetches: if $c = \text{Functor}$, then the task may only apply functorial maps, and thus the only possible outcome is a linear pipeline, e.g. GitHub actions. If $c = \text{Applicative}$, then the results of multiple tasks may be combined together in a static fashion. Hence, the dependency structure is known upfront. The *Make* build system is able to build these kinds of tasks. If $c = \text{Monad}$, the task may bind the result of one query, and decide what other queries to fetch. Hence dependencies may be dynamic.

A collection of tasks $\text{Tasks } c \, k \, v = k \rightarrow \text{Maybe } (\text{Task } c \, k \, v)$ assigns a task to each non-input key, with input keys returning *Nothing*. A build system is then a function that takes a task description and an input store, and produces an updated store.

As a concrete example, consider a spreadsheet where cell $A1$ contains the literal 5, $B1$ contains the formula $A1 + 1$, and $C1$ contains $A1 + B1$. The input keys are cells containing literals ($A1$); the query keys are cells with formulas ($B1, C1$). The *Tasks* function maps $B1$ to a task that fetches $A1$ and returns $A1 + 1$, and $C1$ to a task that fetches both $A1$ and $B1$ and returns their sum. The $C1$ task has two static dependencies — $A1$ and $B1$ — so it type-checks under the $c = \text{Applicative}$ constraint. A task that chose which cell to fetch based on the value of another, e.g. if $A1 > 0$ then $B1$ else $C1$, would require $c = \text{Monad}$.

Given the same *Tasks* description, different build systems produce different update semantics. The paper decomposes build systems along two axes: the *scheduler* (topological, restarting, or suspending) determines the order of task execution, while the *rebuilder* (dirty bits, verifying traces, or constructive traces) determines whether a key needs recomputation. Concrete build systems are instances of this framework:

	Topological	Restarting	Suspending
Dirty bit	Make	Excel	-
Verifying traces	Ninja	-	Shake
Constructive traces	CloudBuild	Bazel	-
Deep constr. traces	Buck	-	Nix

2.1.1 Our formulation

We formalise the framework in Lean 4, making several refinements to the paper’s Haskell presentation. These are not changes to the theory but clarifications that arise naturally when working in a dependently typed host language.

2.1.1.1 Separating inputs from queries

The paper uses a single key type k and distinguishes inputs from computed values by whether *Tasks* returns *Nothing*. We instead separate them at the type level: input keys I with values $V : I \rightarrow \text{Type}$, and query keys Q with results $R : Q \rightarrow \text{Type}$. A task can read inputs via `input i` and fetch query results via `fetch q`, as distinct operations:

```
def Task (α : Type) : Type 1 :=
  ∀ (f : Type → Type) [c f], (∀ i, f (V i)) → (∀ q, f (R q)) → f α
```

Tasks are now total — every query has a task, and *Maybe* is eliminated. The dependent types $V : I \rightarrow \text{Type}$ and $R : Q \rightarrow \text{Type}$ allow different queries to have different result types, which is essential for a type

checker where “what is the type of x ?” and “what are the declarations in file p ?” return values of different types.

2.1.1.2 Well-founded termination

In order to properly define which sets of tasks terminate, we require that the dependency relation on queries (the set of dependency queries a query may fetch) be well founded. This relation is dependent on the set of inputs: given any set of inputs, the relation must be well founded.

For clarity, the parameters are bundled into a record type, that defines the entire configuration:

```
structure BuildConfig : Type 1 where
  I : Type
  V : I → Type
  Q : Type
  R : Q → Type
  rel : (∀ i, V i) → Q → Q → Prop
  wf : ∀ i, WellFounded (rel i)
```

The relation rel is indexed by the input state i : different source files can induce different dependency orders among queries. The task type carries the current input state i_0 and origin query q_0 , and the fetch callback requires a proof that the fetched query q precedes q_0 :

```
def Task (α : Type) : Type 1 :=
  ∀ (f : Type → Type) [c f],
    (∀ i, f (C.V i)) →
    (∀ q, C.rel i_0 q q_0 → f (C.R q)) →
    f α
```

This makes busy evaluation (the naive strategy that always recomputes) provably terminating by well-founded recursion, eliminating the need for runtime cycle detection.

2.1.1.3 Build system structure

A build system is a structure with private state σ , an initialiser, and a build function:

```
structure Build (J : Type) [Input C J] : Type 1 where
  σ : Type
  init : J → σ
  inputs : σ → ∀ i, C.V i
  set : ∀ i, C.V i → StateM σ Unit
  build : Tasks c C → ∀ q, StateM σ (C.R q)
```

The `inputs` field extracts the current input state from the build state, while `set` updates an input value. The separation of inputs from queries enforces by construction that a build system cannot modify its own computed results — only inputs can be set externally.

2.2 Dependent type theory

We implement a dependent type theory in the tradition of Martin-Löf. The reader is assumed to be familiar with the simply typed lambda calculus; we focus on the features that distinguish our system.

2.2.1 Syntax

The core theory maintains a syntactic distinction between *types* and *terms* (Tarski-style universes). The grammar is:

$$\begin{aligned}
 \ell &::= n \mid 0 \mid \text{succ}(\ell) \mid \text{max}(\ell, \ell) && \text{(universe levels)} \\
 A, B &::= \text{Type}_\ell \mid \Pi(x : A).B \mid \text{El}(t) && \text{(types)} \\
 t, s &::= \text{Type}'_\ell \mid x_i \mid c.\{\bar{\ell}\} \mid \lambda(x : A).t \mid ts \mid \Pi'(x : t).s \mid t.k \mid \text{let } x : A := t \text{ in } s && \text{(terms)}
 \end{aligned}$$

Universe levels ℓ form an algebra of named variables n , zero, successor, and binary maximum. Types include universes Type_ℓ , dependent function types $\Pi(x : A).B$, and the decoding operation $\text{El}(t)$ which converts a term-level code into a type. The term formers Type'_ℓ and $\Pi'(x : t).s$ are codes for the corresponding type formers. Variables x_i are de Bruijn indices, $c.\{\bar{\ell}\}$ are global constants applied to universe level arguments, and $t.k$ is projection of the k -th field.

In Russell-style theories (as in Lean 4 itself), types and terms share a single syntactic sort: a Type expression may appear both as a type and as a term. In our Tarski-style formulation, every position in the syntax tree is unambiguously either a type or a term. The decoding operation El bridges the two: if $\Delta; \Gamma \vdash t : \text{Type}_\ell$, then $\text{El}(t)$ is a well-formed type.

Russell-style universes admit a simpler implementation — a single syntactic sort removes the need for El coercions and duplicated type/term formers. We choose Tarski style despite this cost because it admits a cleaner categorical semantics: Type_ℓ is an object and El is a morphism, corresponding directly to the structure of a universe in a category with families. The two styles are known to be equivalent up to translation, but the translation is non-trivial, and the metatheory of Tarski universes is easier to present and reason about formally.

2.2.2 Judgement forms

The theory has six judgement forms, parameterised by a global environment Δ and a local context Γ :

$\vdash \Delta \text{ sig}$	global environment well-formed
$\Delta; \Gamma \vdash$	context well-formed
$\Delta; \Gamma \vdash A \text{ type}$	type well-formed
$\Delta; \Gamma \vdash t : A$	term has type
$\Delta; \Gamma \vdash A \equiv B \text{ type}$	type equality
$\Delta; \Gamma \vdash a \equiv b : A$	term equality

The global environment Δ maps constant names to their definitions and types. The local context Γ is a telescope of typed bindings.

2.2.3 Type formation rules

$$\frac{\Delta; \Gamma \vdash}{\Delta; \Gamma \vdash \text{Type}_\ell \text{ type}} \text{U} \quad \frac{\Delta; \Gamma \vdash t : \text{Type}_\ell}{\Delta; \Gamma \vdash \text{El}(t) \text{ type}} \text{El} \quad \frac{\Delta; \Gamma \vdash A \text{ type} \quad \Delta; \Gamma, x : A \vdash B \text{ type}}{\Delta; \Gamma \vdash \Pi(x : A).B \text{ type}} \Pi \text{-F}$$

2.2.4 Typing rules

$$\frac{\Delta; \Gamma \vdash}{\Delta; \Gamma \vdash x_i : \Gamma(i)} \text{Var} \quad \frac{\Delta; \Gamma \vdash \quad \Delta(c) = A}{\Delta; \Gamma \vdash c.\{\bar{\ell}\} : A} \text{Const}$$

$$\frac{\Delta; \Gamma \vdash A \text{ type} \quad \Delta; \Gamma, x : A \vdash b : B}{\Delta; \Gamma \vdash \lambda(x : A).b : \Pi(x : A).B} \Pi \text{-I} \quad \frac{\Delta; \Gamma \vdash f : \Pi(x : A).B \quad \Delta; \Gamma \vdash a : A}{\Delta; \Gamma \vdash f a : B[a/x]} \Pi \text{-E}$$

$$\frac{\Delta; \Gamma \vdash a : A \quad \Delta; \Gamma, x : A \vdash b : B}{\Delta; \Gamma \vdash \text{let } x : A := a \text{ in } b : B[a/x]} \text{Let} \quad \frac{\Delta; \Gamma \vdash t : A \quad \Delta; \Gamma \vdash A \equiv B \text{ type}}{\Delta; \Gamma \vdash t : B} \text{Conv}$$

The conversion rule allows a term of type A to be given type B whenever A and B are definitionally equal.

2.2.5 Definitional equality

Definitional equality is the congruence closure of the following computation rules:

- β -reduction: $(\lambda(x : A).b) a \equiv b[a/x]$
- δ -reduction: $c.\{\bar{\ell}\} \equiv t$ when $\Delta(c) = t$
- ζ -reduction: $\text{let } x : A := a \text{ in } b \equiv b[a/x]$

- η -conversion: $f \equiv \lambda(x : A). f x$ at function type
- ι -reduction: recursor applied to a constructor computes to the corresponding branch

The full relation includes congruence, symmetry, transitivity, and closure under the conversion rule for types.

2.2.6 Universe polymorphism

Definitions may be parameterised by universe level variables. For example:

```
def id.{u} (α : Type u) (x : α) : α := x
```

At each use site, levels are instantiated explicitly: `id.{0}` or `id.{v}`. There is no universe level inference.

2.2.7 Inductive types

The theory supports user-defined inductive families. An inductive declaration introduces a type, its constructors, and a *recursor*. We illustrate with `Vector`, a length-indexed list:

```
inductive Vector.{u} (α : Type u) : Nat → Type u where
| nil : Vector α Nat.zero
| cons (n : Nat) (head : α) (tail : Vector α n) : Vector α (Nat.succ n)
```

This declaration generates:

- The type `Vector.{u} : (α : Type u) → Nat → Type u`
- Constructors `Vector.nil.{u} : (α : Type u) → Vector α Nat.zero` and `Vector.cons.{u} : (α : Type u) → (n : Nat) → α → Vector α n → Vector α (Nat.succ n)`
- A recursor `Vector.rec.{v, u}` that eliminates a `Vector` by providing a case for `nil` and a case for `cons`:

```
Vector.rec.{v, u} :
  (α : Type u) →
  (C : (n : Nat) → Vector α n → Type v) →
  C Nat.zero (Vector.nil α) →
  ((n : Nat) → (head : α) → (tail : Vector α n) → C n tail → C (Nat.succ n) (Vector.cons α
n head tail)) →
  (n : Nat) → (xs : Vector α n) → C n xs
```

All computation on inductives proceeds through the recursor. There is no primitive pattern matching. The computational rule (*iota-reduction*) fires when the recursor is applied to a constructor — for example, applying `Vector.rec` to `Vector.cons` reduces to the `cons` branch with the fields and recursive result as arguments.

As an example, `Vector.map` applies a function to each element:

```
def Vector.map.{u, v} (α : Type u) (β : Type v) (n : Nat) (f : α → β)
  : Vector.{u} α n → Vector.{v} β n :=
  Vector.rec.{v, u} α (fun k _ => Vector.{v} β k)
    (Vector.nil.{v} β)
    (fun m h _ => Vector.cons.{v} β m (f h))
  n
```

Structures (single-constructor inductives) additionally support *projections* $t.k$, which extract the k -th field from a constructor application.

2.3 Bidirectional type checking

A naive type checker for a dependent type theory must solve the problem of *type inference* for every subterm. This is expensive and in general undecidable for systems with dependent types. *Bidirectional type checking* [2] splits the problem into two mutually recursive judgements:

- **Checking** $\Gamma \vdash e \leq A$: verify that e has type A , where A is already known.
- **Inference** $\Gamma \vdash e \Rightarrow A$: compute the type A of e .

The key insight is that type information flows *downwards* in checking mode and *upwards* in inference mode. In checking mode, the expected type is propagated into the term; in inference mode, the type is synthesised from the term's structure.

2.3.1 Typing rules

The bidirectional presentation of the typing rules in [2] partitions each rule by whether the conclusion is in check or infer mode. The essential rules are:

$$\begin{array}{c}
\frac{}{\Gamma \vdash x_i \Rightarrow \Gamma(i)} \text{Var} \qquad \frac{\Gamma \vdash e \leq A}{\Gamma \vdash (e : A) \Rightarrow A} \text{Anno} \\
\\
\frac{\Gamma, x : A \vdash b \leq B}{\Gamma \vdash \lambda x. b \leq \Pi(x : A). B} \Pi\text{-I} \qquad \frac{\Gamma \vdash f \Rightarrow \Pi(x : A). B \quad \Gamma \vdash a \leq A}{\Gamma \vdash f a \Rightarrow B[a/x]} \Pi\text{-E} \\
\\
\frac{\Gamma \vdash e \Rightarrow B \quad \Gamma \vdash A \equiv B \text{ type}}{\Gamma \vdash e \leq A} \text{Sub}
\end{array}$$

Variables are inferred by lookup (Var). Type ascriptions ($e : A$) switch from check to infer mode (Anno): by annotating a term with a type, the programmer provides the information needed to synthesise. Lambdas are checked against a known Π type (Π -I): the domain is taken from the expected type, so the parameter need not be annotated in the surface syntax. Applications are inferred (Π -E): inferring the function first yields a Π type whose domain is then used to check the argument. The Sub rule bridges the modes: an inferred term may be used in a checking position if its inferred type is convertible with the expected type.

The rules are syntax-directed: each term form has a unique rule applicable in each mode, so the type checker does not backtrack.

2.3.2 Example

Consider checking $(\lambda x. x) \leq \Pi(x : \text{Nat}). \text{Nat}$ in an empty context.

1. The term is a lambda, the expected type is a Π type, so Π -I applies. The domain is Nat and the codomain is Nat . We check the body: $x : \text{Nat} \vdash x \leq \text{Nat}$.
2. The body x is a variable; variables are in infer mode, so we apply Sub to switch from check to infer. First, Var infers $x : \text{Nat} \vdash x \Rightarrow \text{Nat}$. Then Sub requires $\text{Nat} \equiv \text{Nat}$ type, which holds by reflexivity.
3. The derivation closes.

Now consider the term $\lambda x. x$ with no expected type, i.e. $\vdash \lambda x. x \Rightarrow ?$. There is no infer rule for lambdas — the parameter type is not determined by the term. The check must fail unless the programmer provides an annotation, writing $(\lambda x. x) : \Pi(x : \text{Nat}). \text{Nat}$ and applying Anno.

This is why a bidirectional checker is syntax-directed: the need for annotations is entirely localised, and where it arises is predictable from the grammar.

2.3.3 Relation to other approaches

A fully *inference*-based approach à la Hindley-Milner attempts to synthesise a type for every subterm. This works for parametric polymorphism but becomes undecidable or requires metavariables and unification once dependent types are introduced. Systems with implicit arguments (Coq, Agda, Lean) use metavariables and pattern unification to fill in unsynthesisable information.

A fully *checking*-based approach requires the user to annotate every subterm, which is impractical.

Bidirectional typing sits between these extremes: the programmer annotates at a small number of well-defined places (top-level definitions, the boundary of ascribed terms), and the checker propagates the information elsewhere. This keeps the implementation simple while remaining ergonomic.

2.4 Normalisation by evaluation

Deciding definitional equality requires reducing terms to a common form. A straightforward approach is to define reduction rules on the syntax and apply them repeatedly, but this is slow: each reduction step must traverse and rebuild syntax trees.

Normalisation by evaluation (NbE) [3] is a technique that reuses the host language's evaluation mechanism. The idea is to define a *semantic domain* of values, an *evaluation* function from syntax to values, and a *quotation* function from values back to syntax. Definitional equality is then decided by evaluating both sides and comparing the resulting values.

2.4.1 Semantic domain

The semantic domain consists of *values* that represent terms in weak head normal form. Closures — a term paired with its environment — stand for unevaluated function bodies. Neutral terms — a variable or constant head with a spine of eliminators — represent computations that are blocked on an unknown variable.

In our formulation, the syntax uses de Bruijn *indices* (counting from the innermost binder), while values use de Bruijn *levels* (counting from the outermost binder). This avoids the need to shift indices during substitution, which is one of the main sources of bugs and inefficiency in syntactic approaches.

2.4.2 Evaluation and quotation

Evaluation (`eval`) interprets syntax in an environment of values:

- A variable looks up its value in the environment.
- A lambda creates a closure capturing the current environment and the body.
- An application evaluates the function and argument, then performs beta-reduction if the function is a lambda, or extends the spine if it is neutral.
- A constant is left as a neutral term; its definition is only unfolded on demand during *weak head normalisation* (WHNF).

Quotation (`quote`) converts a value back to syntax. For closures, it applies the closure to a fresh variable (creating a new de Bruijn level) and quotes the result. This is where the level-to-index conversion occurs.

2.4.3 Example: open terms and levels

To see why values use levels rather than indices, consider the term $(\lambda f. \lambda x. f\ x)\ g$, where g is a free variable introduced earlier in some outer context (say, at index 0 relative to the outer binder).

Evaluating this term in an empty local environment:

- The outer application evaluates its function $(\lambda f. \lambda x. f\ x)$ to a closure $\langle [], \lambda x. f\ x \rangle$ capturing an empty environment, and its argument g to the value corresponding to the outer free variable — a neutral term $\langle \text{Var } 0, \text{nil} \rangle$ at level 0.
- Beta-reduction extends the environment of the closure with the neutral g and evaluates the body $\lambda x. f\ x$, producing another closure $\langle [g], f\ x \rangle$.

To quote this closure back to syntax, we need to look under the binder. `quote` introduces a fresh variable at level 1 (since g occupies level 0), extends the environment with it, and evaluates the body:

- In the extended environment $[g, x_1]$, the body $f\ x$ evaluates: f is looked up at index 1 (the second-to-last entry) and is the neutral g at level 0; x is looked up at index 0 and is the neutral x_1 at level 1.
- The application of two neutrals produces a neutral whose head is g and whose spine is $[x_1]$.

Quoting this neutral converts the levels back to indices relative to the current scope depth 2: g at level 0 becomes index 1; x_1 at level 1 becomes index 0. The final result is $\lambda x. g\ x$, correctly expressed in terms of the outer variable g .

The crucial property is that g 's level stayed fixed throughout — it was level 0 when introduced, and still level 0 at the end. Under a de Bruijn index representation for values, g would have needed to be shifted every time a new variable was pushed onto the environment. Because values use levels, the representation of g is stable, and only the final quote step needs to convert to indices.

2.4.4 Conversion checking

To check whether two terms are definitionally equal, we evaluate both to values and compare them structurally. When both sides are neutral with the same head, we compare their spines. When one or both sides are headed by a defined constant, we unfold (*delta-reduce*) and continue. When both sides are lambdas, we apply both to a fresh variable and compare the bodies.

2.5 Related work

2.5.1 Incremental elaborators

Fredriksson's *sixty* [4] is an incremental elaborator for a dependently typed language, built on *Rock*, a Haskell library inspired by the same "Build Systems à la Carte" framework. It demonstrates that query-based incrementality is viable for dependent type checking, and is the closest prior work to this project. The key architectural difference is that *sixty* is implemented in Haskell, where lazy evaluation enables *glued* values that carry both folded and unfolded representations simultaneously.

Rust-analyzer uses *Salsa*, a query-based incremental computation framework inspired by the same paper, for incremental type checking of Rust. *Salsa* uses a monotonic revision clock and dependency tracking to avoid recomputation. Our Shake-based approach follows a similar design.

2.5.2 Efficient elaboration

Kovacs's *smalltt* [5] demonstrates that high-performance elaboration is achievable through careful attention to evaluation strategies. It combines higher-order abstract syntax (HOAS), glued evaluation, and approximate conversion checking to achieve type-checking speeds that significantly outperform production systems on benchmarks. Several of these techniques — particularly approximate conversion checking with rigid/flex/full modes — directly influenced the design of our conversion checker.

2.5.3 Lean 4

Lean 4 [6] is the primary reference for the surface language and inductive type machinery. Our elaborator supports the same declaration forms (`def`, `inductive`, `structure`, `axiom`) and the same recursor-based elimination. The core theory departs from Lean's kernel in using Tarski-style rather than Russell-style universes, and in omitting metavariables and implicit arguments entirely.

2.6 Starting point

Prior to starting the project, I had experience implementing type checkers for simply typed languages, but had not implemented a dependently typed elaborator or a build system. I was already familiar with type theory and had prior experience working on the OCaml compiler.

No implementation code was written before the project started. The project was initially planned in Rust, but an early switch was made to OCaml, and then to Lean 4. These transitions occurred during the project and are discussed in the Implementation chapter. No existing codebases were used as a basis; several projects were consulted as references, including *smalltt*, *sixty*, and Lean 4's source.

3 Implementation

This chapter describes the implementation of the elaborator. We follow the pipeline from source text to elaborated output: parsing, bidirectional type checking, normalisation by evaluation, conversion checking, inductive type elaboration, and the incremental build architecture that ties them together.

The implementation is written in Lean 4. Throughout this chapter, code snippets are excerpts from the implementation.

3.1 Architecture

We structure the elaborator as a collection of cooperating components. Source text is parsed by a hand-rolled monadic Pratt parser (Section) into a concrete syntax tree and a desugared abstract syntax tree. The AST is elaborated by a bidirectional type checker (Section) that uses normalisation by evaluation (Section) to reduce terms and a conversion checker (Section) to decide definitional equality. Inductive declarations (Section) are elaborated into types, constructors, and recursors. All of this is decomposed into fine-grained queries memoised by a Shake-based build system (Section), and diagnostics and hover information are served to a VSCode extension via the Language Server Protocol (Section).

The core of the elaborator is the `ElabM` monad:

```
abbrev ElabM :=
  Task Monad config ι₀ q₀
  |> StateT ElabState
  |> ReaderT ElabContext
```

Here `config` : `BuildConfig` bundles the input and query types, while `ι₀` and `q₀` are the current input state and origin query carried along for the well-founded dependency relation.

Each transformer serves a distinct purpose:

- `Task` is the incremental query monad. Fetches through `Task.fetch` register dependencies in the Shake graph.
- `StateT ElabState` threads mutable elaboration state: the local environment of constants elaborated so far, a cache of fetched constants, and accumulated diagnostics and hover information.
- `ReaderT ElabContext` carries read-only context: the file path, the current declaration name, the universe parameter list of the current definition, and a path into the AST for diagnostic positions.

An earlier iteration of the design used `WriterT ElabInfo` instead of threading diagnostics through `ElabState`. Microbenchmarks showed this introduced significant overhead: every monadic `>>=` invoked the monoid `append` on the `ElabInfo` tuple, even when nothing was emitted. Replacing `WriterT` with a push-only interface over `StateT` arrays produced a measurable speedup on repeated-conversion benchmarks.

The components are connected by query dependencies, not by direct function calls. The parser's output is the `Val` of a `Key.cst` query; the AST is the `Val` of a `Key.ast` query; each declaration's elaborated form is the `Val` of a `Key.elabDecl` query, which depends on `Key.constant` queries for every referenced name. This structure makes the system incremental: when an input changes, the build system walks the dependency graph backward and invalidates exactly the queries that transitively depended on it.

3.2 Parsing

The parser is a hand-rolled recursive descent Pratt parser, implemented as monadic combinators over `StateT State (Except ParseError)`. Parsing proceeds in two stages: the source text is first parsed to a concrete syntax tree (`Cst`) that retains trivia and exact token widths, and the CST is then desugared to an abstract syntax tree (`Ast`) that strips trivia and expands surface syntax.

The parser itself is unremarkable — every language needs one, and Pratt parsing is a well-understood technique for operator precedence. The interesting design choice is the representation of the CST.

3.2.1 Green trees

The CST follows the *green tree* pattern (used by Roslyn and rust-analyzer). Nodes and tokens are tagged by `SyntaxNodeKind`, and positions are not stored explicitly:

```
inductive Cst : Type
  | token (kind : SyntaxNodeKind) (val : String)
  | node (kind : SyntaxNodeKind) (children : Array Cst)
with
```

```
@[computed_field]
width : Cst → Nat
| .token _ val => val.length
| .node _ children => children.map width |>.sum
```

Each node carries only its kind and children; its *width* is a computed field summing the widths of its descendants. A source position is then reconstructed by walking from the root and summing the widths of preceding children. Because nodes do not store absolute positions, two identical subtrees are structurally equal regardless of where they appear in the file — the CST is Hashable, which lets the build system short-circuit recomputation when a file is edited in a way that does not affect a given subtree.

This matters for incrementality at the level *above* parsing. Although the parser itself is not incremental (a full reparsing on each edit is cheap), the memoised `Key.cst` query is keyed by file content, and downstream queries that depend on parsed subtrees benefit from structural equality of the CST. For example, the `Key.declarationIndex` query only depends on the kinds and names of top-level declarations; if an edit within one declaration does not change the enclosing structure, the declaration index’s hash is unchanged and dependent queries are not invalidated.

3.2.2 Language server integration

Alongside the CST, parsing produces a `SourceMap` that bidirectionally maps paths in the CST to paths in the AST. A *path* is a list of child indices from the root, so the root of either tree has path `[]`, its first child has path `[0]`, and so on. The bidirectional map is:

```
structure SourceMap where
  cstToAst : HashMap Path Path
  astToCst : HashMap Path Path
```

The map is populated during desugaring: as each AST node is constructed from a CST node, an entry is added to both tables. The language server uses this in both directions:

- *Hover*: given a cursor position, the CST is walked to find the path of the token under the cursor, then `cstToAst` gives the AST path. The elaborator’s hover information is keyed by AST path, so the correct hover content is retrieved directly.
- *Diagnostics*: the elaborator records diagnostics at AST paths. `astToCst` maps these back to CST paths, which are then converted to source spans by summing widths.

Without the source map, a position-based language server would need to traverse the entire AST comparing source ranges at every node. The path-based lookup is $O(\text{depth})$ on the tree rather than $O(\text{size})$.

3.2.3 Error recovery

The parser uses lightweight error recovery: on encountering a parse error, a diagnostic is emitted and the parser attempts to continue at the next top-level declaration boundary (`def`, `inductive`, `axiom`, etc.). This ensures that a single malformed declaration does not prevent the rest of the file from being elaborated — the language server continues reporting errors throughout the file.

A separate converter from `Lean.Syntax` to `Ast` is available via the metaprogramming framework, for writing inline test cases in Lean source files. This is not used by the main pipeline.

3.3 Bidirectional type checking

Type checking is bidirectional. Instead of a single judgement $\Gamma \vdash t : A$ that both produces and checks types, the algorithm is split into two mutually recursive functions:

- `inferTm ctx ast : OptionT ElabM (Tm n × VTy n)` — given an AST, synthesise a core term and its type. Used when the expected type is not known or would be recomputed redundantly.

- `checkTm ctx expectedTy ast : ElabM (Tm n)` — given an AST and an expected type, produce a core term of that type. Used when the expected type is already available from the surrounding context. Failure is handled internally by emitting a diagnostic and inserting a fresh axiom as a placeholder.

The split has three benefits. First, many terms can be checked without needing to synthesise their types: a lambda’s parameter type is determined by the expected Pi type, so the parameter can be left unannotated. Second, the algorithm is syntax-directed — each AST constructor dispatches to a specific case, avoiding the need for backtracking or unification. Third, errors are pinpointed: a mismatch between the expected and inferred type is reported at the exact subterm, rather than propagating upward.

The mode-switching rules are standard:

- **Lambdas** are checked. In checking mode against expected type $\Pi(x : A).B$, the body is checked against B in the context extended with $x : A$. If the AST’s parameter has a type annotation, it must match A (via conversion checking); otherwise A is taken from the expected type.
- **Applications** are inferred. The function f is inferred, yielding a type T . If T is not a Pi type after weak head normalisation, the elaborator fails with a type error. Otherwise $T = \Pi(x : A).B$ and the argument a is checked against A ; the result has type $B[a/x]$.
- **Variables** are inferred by looking them up in the local context or the global environment.
- **Let-bindings** are inferred by first inferring the bound value’s type, then checking the body in the extended context.
- **Universes** Type_ℓ are inferred to have type $\text{Type}_{\ell+1}$.
- **Subsumption**: when an inferred term is used in a checking position, the inferred type is compared against the expected type by the conversion checker.

The core loop is `inferTm/checkTm`, mutually recursive with a helper `checkIdent` that handles variable lookup (checking the local context first, then the global environment). For example, the application case of `inferTm` is:

```
| .node `Term.app #[fn, arg] => do
  let (fnTm, fnTy) ← inferTm ctx fn
  let .pi _ dom (env, codTm) := fnTy
  | raiseError (.expectedFunctionType ctx.names (← fnTy.quote))
  let argTm ← checkTm ctx dom arg
  let argVal ← argTm.eval ctx.env
  let codVal ← codTm.eval (env.cons argVal)
  return (.app fnTm argTm, codVal)
```

The function is inferred, yielding a type `fnTy`. Because evaluation eagerly unfolds constants into glued values, and `inferTm` always returns a `VTy` produced by `Ty.eval`, `fnTy` is already in weak head normal form — no explicit `whnf` call is needed to expose the Pi structure. The argument is then checked against the domain, and the codomain is substituted via evaluation in the closure’s extended environment, rather than by explicit syntactic substitution.

When checking or inference fails — due to a sorry, an unbound variable, or a type mismatch — the elaborator emits a diagnostic and inserts a fresh axiom as a placeholder at the expected type. This keeps the rest of the file type-checkable, so the language server can continue reporting errors throughout the file rather than stopping at the first mistake.

3.3.0.1 Comparison with unification-based elaboration

Pure type inference (synthesising types for every subterm) is incomplete for dependent type theories: without additional structure, variables like `fun x => x` have no principal type. Systems with implicit arguments and metavariables fill this gap via unification, solving for the missing types during elaboration. This is powerful but introduces substantial complexity — pattern unification, occurs checks, postponed constraints, meta freezing — none of which appear in this implementation.

Bidirectional type checking [2] sidesteps this by requiring the user to annotate just enough types that checking is deterministic. In practice the annotation burden is small: top-level definitions have types (because they are either explicit or inferred from a body), lambdas under a Π inherit their domain, and let-bindings propagate their inferred type to the body. The cases that require annotations — such as the binder type of a top-level lambda with no outer context — are exactly the cases where the user would need to provide the information anyway.

The approach taken here is syntax-directed with no metavariables and no backtracking. The trade-off is that the surface language is more verbose than Lean’s or Agda’s, but the elaborator is significantly simpler, which was a deliberate choice given the scope of this project.

3.4 Normalisation by evaluation

3.4.1 Semantic domain

The semantic domain separates *values* from *syntax*. Values (VTm , VTy) represent terms in weak head normal form, using de Bruijn *levels* rather than indices:

```
inductive VTm : Nat → Type
| u'      : Universe → VTm n
| neutral : Neutral n → VTm n
| lam     : Name → VTy n → ClosTm n → VTm n
| pi'     : Name → VTm n → ClosTm n → VTm n
| glued   : Neutral n → Tm 0 → VTm n
```

A `neutral` is a head (variable or constant) applied to a spine of eliminators. A `lam` carries a closure — an environment $Env\ n\ m$ paired with a body term $Tm\ (m + 1)$ having one free variable. Beta-reduction evaluates the body in the environment extended with the argument. A `glued` value carries both a neutral (the folded form, for quotation) and the unevaluated definition body (for reduction).

The choice of de Bruijn levels for values and indices for syntax is deliberate. Under levels, a variable’s name is absolute rather than relative, so weakening (embedding a value into a larger scope) is a runtime no-op: the level remains the same when additional variables are pushed onto the environment. Under indices, every weakening requires traversing the value to shift every variable. Since NbE weakens values frequently — closures are opened at fresh levels during quotation and conversion — this would be a significant cost. The scope index n in $VTm\ n$ enforces well-scoping at the type level; weakening then becomes a proof obligation that omega discharges automatically, and the underlying data is reused via `unsafeCast`.

3.4.1.1 Choice of closure representation

Closures in NbE can be represented in two ways:

- **Higher-order abstract syntax (HOAS):** a lambda carries a host-language function $VTm \rightarrow VTm$. Beta-reduction is a function call; the host compiler’s closure optimisations apply.
- **Defunctionalised closures:** a lambda carries the body as syntax paired with its captured environment. Beta-reduction re-interprets the body by evaluating it in the extended environment.

HOAS is significantly faster in practice: `smalltt` [5] reports a $2\text{--}3\times$ speedup over defunctionalised closures on identical benchmarks, because GHC compiles the host function to efficient native code. However, HOAS is strictly positive only in languages that permit non-positive inductive types. Lean’s kernel rejects $VTm \rightarrow VTm$ in the `lam` constructor, as it violates positivity and opens the door to non-termination via Curry’s paradox.

We could bypass the check with `unsafe inductive`, as the normalisation benchmarks in Lean themselves do. This was considered, but rejected: marking VTm unsafe would prohibit proving any properties of the evaluator, which in turn would prohibit proving the correctness of conversion checking and elaboration. The defunctionalised representation keeps the evaluator within the kernel’s logic.

3.4.2 Evaluation

Evaluation (`Tm.eval`) interprets syntax in an environment of values. The environment is indexed by the scope size, matching the scope index of the syntax being evaluated:

```
partial def Tm.eval {n c} : Tm c → SemM n c (VTm n)
| .u' i          => return .u' i
| .var i         => return (← read).get i
| .const name us => do
  let some tm ← fetchDefinition name
  | return .neutral (.const name us, .nil)
  let some info ← fetchConstantInfo name
  | return .neutral (.const name us, .nil)
  return .glued (.const name us, .nil)
  (tm.substLevels (info.univParams.zip us))
| .lam x a body => return .lam x (← a.eval) (← read, body)
| .app fn arg   => do (← fn.eval).app (← arg.eval)
| .pi' x a b    => return .pi' x (← a.eval) (← read, b)
| .proj i t     => do (← t.eval).proj i
| .letE _ _ t b => do b.eval (.cons (← t.eval) (← read))
```

The cases are:

- **Variables** are looked up in the environment by de Bruijn index.
- **Constants** evaluate to *glued* values when a definition is available: the result carries both the folded neutral form and the unevaluated definition body (with universe parameters substituted). If the constant has no definition (e.g. an axiom or an opaque declaration), a plain neutral is returned.
- **Lambdas** create closures capturing the current environment and the unevaluated body. No reduction happens under the binder.
- **Applications** evaluate both sides, then dispatch on the function's head in `VTm.app`: beta-reduction if it is a lambda, extending the spine if it is neutral or glued.
- **Let-bindings** evaluate the bound term and extend the environment, then evaluate the body.

Weak head normalisation (`VTm.whnf`) unfolds further on demand: for a constant-headed neutral, it delta-reduces (fetches and evaluates the definition) and re-applies the accumulated spine. For a glued value, it forces the unfolded body. It also performs iota-reduction when a fully-applied recursor has a constructor as its major premise.

3.4.3 Quotation

Quotation (`VTm.quote`) converts values back to syntax. For closures, it applies the closure to a fresh variable at the current de Bruijn level and quotes the result, converting levels to indices during the traversal. For `VTm.glued` nodes, quotation uses the folded form (the neutral), producing compact output that preserves top-level definition names rather than inlining their bodies.

3.5 Conversion checking

Conversion checking decides definitional equality of two values. The naive algorithm is: reduce both sides to normal form, then compare structurally. This is correct but slow, because it forces all reducible subterms regardless of whether comparison actually needs them.

The approximate algorithm used here, inspired by Kovacs's `smalltt` [5], bounds the amount of speculative work by a three-state mode parameter:

```
inductive ConvState where
| rigid
| flex
| full
```

The three modes are:

- **Rigid**: the default mode, used at the top level and under canonical formers (lambdas, Pi types, constructors). When two neutral or glued values have the same defined head, spines are compared speculatively in flex mode. If the flex comparison succeeds, no unfolding was needed — the two terms are structurally equal at the folded level. If it fails, both sides are unfolded to their definitions and compared in full mode.
- **Flex**: entered only from rigid's speculative spine check. No definitions are unfolded; no fallback is attempted. If heads agree, the spines are compared elementwise, still in flex mode. Any mismatch causes immediate failure, returning control to the rigid caller, which then commits to unfolding.
- **Full**: entered from rigid after a flex failure. All definitions are unfolded immediately on encounter. Once in full mode, all recursive calls remain in full.

This ensures at most one backtrack per subterm: rigid attempts a cheap flex check, and on failure commits to the expensive full check. Because flex never unfolds, the cost of a failed speculation is bounded by the size of the matching prefix of the two spines.

To see the benefit, consider comparing $f(g(h\ x))$ with $f(g(h\ x))$ where f, g, h are top-level definitions. The naive algorithm unfolds f, g, h on both sides and compares the unfolded bodies — potentially an exponentially large computation if the definitions are deeply nested. The approximate algorithm observes that the two terms have the same folded structure: same head f , same spine. The flex comparison descends into the argument, observes the same head g , and so on, bottoming out at x with no unfolding at all. The check runs in time proportional to the syntactic size of the folded term, not the unfolded term.

For eta-conversion, two cases are handled: function eta, where $f \equiv \lambda x. f\ x$ is decided by opening both sides at a fresh variable and comparing the bodies; and structure eta, where $c(r_1, \dots, r_k) \equiv s$ (with c a constructor of a single-constructor inductive) is decided by comparing each projection $s.i$ with the corresponding field r_i .

3.5.0.1 Comparison with naive reduction

A naive conversion checker would reduce both sides to normal form and compare. This has two problems. First, normal forms can be exponentially larger than the original terms: consider a definition `def big := ...` that unfolds to a deeply nested expression; comparing `big` with itself would first materialise the huge normal form, then walk it structurally, even though the syntactic equality `big = big` is immediately decidable. Second, the comparison walks the entire normal form even when a mismatch is present near the surface, wasting work.

The rigid/flex/full algorithm addresses both problems. Flex mode exploits the observation that most conversion checks succeed on identical or structurally similar terms: a flex comparison succeeds without any unfolding, running in time proportional to the syntactic size. When flex fails, the algorithm commits to full unfolding, but only for the specific subterm where speculation failed — not for the whole expression. The bounded backtracking ensures that the asymptotic cost is at most the minimum of the two strategies: the flex cost if speculation succeeds, or the flex cost plus the full cost if it fails.

An alternative design, used in some systems, is *on-the-fly* reduction: weakly head normalise each side just enough to decide equality at each level, and recurse under binders. This avoids eagerly computing full normal forms, but without the rigid/flex distinction it still unfolds aggressively whenever the heads do not syntactically match. The speculative flex check avoids this unfolding in the common case where heads are the same defined constant.

3.6 Inductive types

Inductive declarations are the means by which the user extends the theory with new data types. A declaration

```

inductive I.{u} (P : Type u) : Nat → Type u where
| c1 : ... → I P n1
| c2 : ... → I P n2

```

introduces the *type former* I , a family of *constructors* c_i , and a *recursor* $I.rec$. Each constructor is a term that builds values of I at specific indices; the recursor is the unique non-dependent eliminator that computes on each constructor.

The elaboration process proceeds in several phases:

1. **Elaborate the type.** Parameters and indices are elaborated in order. The result type must be a universe Type_u . Note that the distinction between parameters and indices is syntactic: parameters appear before the $:$, indices after.
2. **Register the inductive as opaque.** Before elaborating constructors, the inductive is added to the global environment as an opaque declaration with just its type. This lets constructors refer to the inductive itself during their own elaboration (via the constant name), without creating a circular dependency.
3. **Elaborate each constructor.** Each constructor's fields are elaborated in a context that binds a variable I representing the inductive (not the actual constant, which is not yet fully registered). The constructor's result type must be I applied to the inductive's parameters *verbatim* — the same de Bruijn indices as the parameter telescope — followed by chosen indices. This constraint (`ctorParamMismatch` on mismatch) simplifies the recursor generation, which relies on parameters being invariant across constructors.
4. **Check strict positivity.** In each field's type, the inductive may appear only in *strictly positive* positions. Non-positive occurrences are rejected as `nonPositiveOccurrence` errors. The positivity analysis permits *nested* recursion: a field may be a function type whose result mentions the inductive, provided the inductive does not appear in any argument type of that function.
5. **Check universe consistency.** Each field's universe level must be bounded by the inductive's result universe, rejecting `fieldUniverseTooLarge` otherwise. This ensures the inductive type lives in a universe large enough to contain all its fields.
6. **Generate the recursor type.** The recursor takes the parameters, a *motive* $C : \text{indices} \rightarrow I \text{ parameters indices} \rightarrow \text{Type}_v$, one *minor premise* per constructor (which produces C applied to the constructor), the indices, and finally the major premise of type I . The motive's universe v becomes an additional universe parameter of the recursor (see below).
7. **Generate the recursor rules.** One rule per constructor: when $I.rec$ is applied to c_k args, it reduces to the k -th minor premise applied to args. Recursive fields (where the inductive appears in the field's type) additionally contribute *induction hypotheses* computed by recursive calls to $I.rec$.
8. **Replace the inductive entry.** Finally, the opaque placeholder registered in step 2 is replaced with the full inductive declaration, including the list of constructor names.

For example, the recursor for `Nat`:

$$\text{Nat.rec}\{u\} : (C : \text{Nat} \rightarrow \text{Type}_u) \rightarrow C \text{ zero} \rightarrow ((n : \text{Nat}) \rightarrow C n \rightarrow C (\text{succ } n)) \rightarrow (n : \text{Nat}) \rightarrow C n$$

with the *iota*-reduction rules:

$$\begin{aligned} \text{Nat.rec } C \ z \ s \ \text{zero} &\rightsquigarrow z \\ \text{Nat.rec } C \ z \ s \ (\text{succ } n) &\rightsquigarrow s \ n \ (\text{Nat.rec } C \ z \ s \ n) \end{aligned}$$

Structures (single-constructor inductives) additionally generate *projection* functions $I.f_k$, one per field, that extract the k -th field from a constructor application. The conversion checker includes a structural eta-expansion rule for structures: $c \ (s.f_1) \ \dots \ (s.f_k)$ is identified with s .

3.6.0.1 Universe handling

Universe polymorphism makes recursor generation subtle: the motive’s universe may differ from the inductive’s universe. For example, elimination from `Nat` into `Type 0` gives ordinary recursion, while elimination into `Type (u+1)` gives large elimination.

The elaborator handles this by generating a fresh universe parameter `motiveUnivName`, distinct from all universe parameters of the inductive itself, and prepending it to the recursor’s universe parameter list. So for a monomorphic inductive like `Nat` (`univParams = []`), the recursor has `recUnivParams = [motiveUnivName]`. For `List.{u}` (`univParams = [u]`), the recursor has `recUnivParams = [motiveUnivName, u]`. At each use site, `Nat.rec.{v}` instantiates the motive’s universe to `v`.

The freshness is ensured by `Universe.freshName`, which picks a name not in the inductive’s universe parameter list. The convention is that the motive’s universe is the *first* universe parameter of the recursor.

3.6.0.2 Recursors rather than pattern matching

Modern proof assistants like Lean, Agda, and Coq compile user-facing pattern matching to recursor applications internally. Pattern matching is more convenient for users, but significantly more complex to implement: case trees, catch-all patterns, dependent case analysis, and coverage checking all add machinery.

The core theory here uses recursors directly, as Coq did originally. This keeps the core simple and the trusted code base small: the elaborator need only check that constructors are well-typed and generate the recursor; no pattern-matching compiler is needed. The cost is ergonomics — `stdlib` definitions are written with explicit recursor calls rather than `match` — but the core theory is small enough that adding a pattern-matching frontend would be straightforward future work.

3.7 Incremental elaboration

Elaboration is decomposed into queries managed by a Shake-based build system. Each query is identified by a tag and parameters, and returns a value whose type is determined by the tag. The query types are defined as a pair of indexed inductive types. The actual `Key` type has 17 cases; the principal ones are:

```
inductive Key where
| cst : FilePath → Key
| ast : FilePath → Key
| declarationIndex : FilePath → Key
| elabCmdAt : FilePath → Nat → Key
| elabDecl : FilePath → Name → Key
| constant : FilePath → Name → Key
| lookupInfo : FilePath → Name → Key
-- ... others omitted

def Val : Key → Type
| .cst _      => Cst × Array ParseError
| .ast _      => Ast
| .declarationIndex _ => HashMap Name Nat × Array Diagnostic
| .elabCmdAt _ _    => Global × ElabInfo
| .elabDecl _ _     => Option (Constant × Origin) × ElabInfo
| .constant _ _     => Option (Constant × Origin)
| .lookupInfo _ _   => ElabInfo
-- ... others omitted
```

The dependent typing of `Val` is essential: a single query map cannot be used to store results of different types without either existential packing or a universal result type. Using a type-level function, each query’s result type is precisely determined by its key.

The principal queries are:

- `declarationIndex`: given a file path, returns a map from declaration names to their indices within the file. This query depends only on the parsed AST, so it is recomputed only when the file is edited.
- `elabDecl`: elaborates a single declaration. It fetches the AST, looks up the declaration by name, and type-checks it. It depends on constant queries for every constant it references.
- `constant`: looks up the elaborated form of a named constant. This is the primary cross-declaration dependency.

When the elaborator encounters a reference to a constant, it calls `fetchConstant`, which issues a `Key.constant` query through the build system. This registers a dependency: if the referenced constant changes, the current query is invalidated and recomputed on the next build.

Within a single elaboration run, an `entryCache` in `ElabState` memoises constant lookups to avoid repeated queries to the build system for the same name.

3.7.0.1 Granularity of queries

The choice of query granularity trades off overhead against incremental precision. Too coarse — e.g. one query per file — and any edit invalidates the whole file. Too fine — e.g. one query per expression — and the per-query overhead dominates, making the system slower than batch elaboration for typical edits.

The granularity here is per-declaration. A `Key.elabDecl` query elaborates one top-level `def`, `inductive`, or `structure`. When a declaration is edited, only that declaration’s query is invalidated; dependent declarations are recomputed only if the edit changes the declaration’s cached `Constant` (its elaborated type and body). This matches the granularity at which a user typically edits: they change one definition at a time, and expect the feedback loop to scale with the cost of that definition, not the whole file.

Finer granularity — for example, caching the elaborated form of individual subterms — would require treating each subterm as a named entity, which it is not in the source language. Implementing this would require either heuristic naming (hashing the subterm’s syntax) or restructuring the core theory to make subterms addressable. Neither is compelling for the cost.

3.7.0.2 Dependently-typed query results

An alternative to the dependent type function `Val : Key → Type` is to use a single result type that subsumes all query results, e.g. a sum type `inductive Result | parsed : ... | elaborated : ... | ...` or an existential `Σ α, α`. Both have drawbacks:

- A sum type forces every query to return a tagged value and every consumer to pattern-match and handle an “impossible” case when the tag does not match. This is boilerplate that the dependent typing eliminates.
- An existential type erases the result type entirely, forcing `unsafeCast` or a proof-carrying wrapper at every fetch site. Neither integrates cleanly with the elaborator’s other structures.

The dependent `Val` makes `fetch (Key.constant p n) : Task (Option Constant)` type-check without any tagging, and the compiler can specialise each call site to the specific result type. This is a natural use of Lean’s dependent types that has no direct counterpart in Haskell’s `Rock` or `Salsa` implementations, where singleton types or type families are needed to approximate the same pattern.

3.8 Language server

The elaborator doubles as a language server. During elaboration, hover information and diagnostics are collected in `ElabState` and returned alongside the elaboration result. A VSCode extension communicates with the elaborator via the Language Server Protocol, providing:

- **Diagnostics**: type errors, unbound variables, and universe mismatches, positioned at the relevant source location via path indices into the AST.
- **Hover information**: the type of the term under the cursor, or the full signature of a referenced constant.

4 Evaluation

We evaluate the elaborator along three axes: correctness, performance, and incrementality.

4.1 Correctness

The primary correctness test is the standard library. The `stdlib` contains 2,200 lines of qdt code across 36 files, covering natural number arithmetic, propositional equality, well-founded recursion, sigma types, monadic abstractions, and the Ackermann function. Each definition is fully explicit — no implicit arguments or wildcards — and type-checked from scratch on every run.

Equality proofs in the `stdlib` serve as integration tests for the conversion checker: a proof `Eq.refl.{0} Nat 6` at type `Nat.add 2 4 = 6` succeeds only if the elaborator correctly reduces `Nat.add 2 4` to 6 via *iota*-reduction on the recursor.

4.2 Batch elaboration performance

We measure the time to elaborate the full `stdlib` from a cold start, with no cached results.

Cold-start elaboration of the full `stdlib` completes in approximately 438ms (median of three runs: 447ms, 408ms, 438ms).

4.3 Conversion checking performance

To stress-test the evaluator and conversion checker in isolation, we use Church-encoded benchmarks adapted from Kovacs’s normalisation-bench [5]. These encode natural numbers and binary trees as lambda terms, then check definitional equality between two independently computed values of the same normal form.

The benchmarks are:

- **Nat n conv**: check that two Church numerals, both representing n , are definitionally equal. This forces n beta-reductions on each side.
- **Tree n conv**: check that two full binary trees of depth n (with $2^{n+1} - 1$ nodes) are definitionally equal.

`Nat 10K conv` (checking equality of two Church numerals representing 10,000) completes in approximately 136ms (median of three runs: 139ms, 134ms, 136ms). Larger benchmarks (`Nat 100K` and above) overflow the stack. This is a known limitation of the defunctionalised closure representation used in the evaluator — each beta-reduction is a recursive call, and Church numerals with n successors require n stack frames.

For comparison, `smalltt` handles `Nat 5M conv` in approximately 90ms and `Nat 10M conv` in 180ms, using HOAS closures compiled by GHC. The stack overflow in our implementation prevents direct comparison at these scales.

4.4 Incremental re-elaboration

The key benefit of the query-based architecture is that editing a single definition does not require re-checking the entire file. We measure the time to re-elaborate after modifying a definition, compared to batch re-elaboration.

TODO: table comparing incremental vs batch times for targeted edits.

4.5 Discussion

TODO

5 Conclusion

5.1 Summary

We have presented an incremental, query-based elaborator for a dependently typed language, implemented in Lean 4. The elaborator supports dependent function types, a hierarchy of Tarski-style universes with universe polymorphism, and inductive types with recursors. Elaboration is decomposed

into fine-grained queries managed by a Shake-based build system, so that editing a single definition recomputes only the queries that depend on it.

The build system framework is formalised in Lean 4, refining the “Build Systems à la Carte” framework with a type-level separation of inputs from queries and well-founded termination of the dependency relation. The elaborator itself uses normalisation by evaluation for efficient term reduction and an approximate conversion checker with rigid/flex/full modes to bound backtracking.

TODO: summarise evaluation results.

5.2 Future work

Several directions remain open:

- **Metavariables and implicit arguments.** The current system requires all terms to be fully explicit. Adding metavariables with pattern unification would make the surface language significantly more usable.
- **Parallelism.** The build system framework supports a base monad parameter that could be instantiated with IO for parallel query evaluation. This would allow independent queries to be elaborated concurrently.
- **Cancellation.** A language server should be able to cancel in-progress elaboration when the user edits the file again. The middleware framework could support this via an exception monad that preserves partial progress in the memo store.

Bibliography

- [1] A. Mokhov, N. Mitchell, and S. Peyton Jones, “Build Systems à la Carte,” *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, Sept. 2018, doi: 10.1145/3236774.
- [2] J. Dunfield and N. Krishnaswami, “Bidirectional Typing,” *ACM Computing Surveys*, vol. 54, no. 5, 2021, doi: 10.1145/3450952.
- [3] A. Abel, “Normalization by Evaluation: Dependent Types and Impredicativity,” *Lecture notes*. 2013.
- [4] O. Fredriksson, “sixty.” [Online]. Available: <https://github.com/ollef/sixty>
- [5] A. Kovács, “smalltt.” [Online]. Available: <https://github.com/AndrasKovacs/smalltt>
- [6] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, Springer, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.